Documentation technique

VITE & GOURMAND

Miguel Arteaga

16/11/2026

Documentation technique

1. Réflexions initiales et choix technologiques

Avant le développement de l'application Vite & Gourmand, plusieurs choix technologiques ont été réalisés afin de répondre aux exigences du cahier des charges.

Front-end

- HTML5 : utilisé pour structurer les différentes pages de l'application dans le respect des critères d'accessibilité RGAA.
- CSS3 modulaire & Responsive Design : utilisé pour appliquer la charte graphique définie, créer les composants visuels et garantir une adaptation complète aux ordinateurs, tablettes et smartphones.
- JavaScript (ES6+ natif) : utilisé pour dynamiser l'interface sans rechargement complet de la page (filtrage multicritère des menus en temps réel, calcul dynamique de la distance routière et du tarif de commande, carrousels de photos et d'avis, modales accessibles, navigation par onglets dans les espaces Client, Employé et Administrateur).

Back-end & Architecture logicielle

- PHP 8.2 en Programmation Orientée Objet (POO) : typage strict des propriétés, des arguments et des valeurs de retour, encapsulation des données et chargement automatique des classes selon le standard PSR-4 (App\ → app/) via Composer.
- Architecture MVC (Modèle - Vue - Contrôleur) avec Contrôleur Frontal unique :
    - Contrôleur Frontal (public/index.php & App\Core\Router) : point d'entrée unique de toutes les requêtes HTTP (GET et POST), assurant le routage, le contrôle d'accès par rôle et la validation des jetons anti-CSRF.
    - Modèles (app/Models/) : 11 classes métier représentant fidèlement les entités du domaine avec attributs privés et accesseurs.
    - Pattern Repository (app/Repositories/) : 11 classes d'accès aux données isolant intégralement les requêtes SQL et NoSQL du reste de l'application, avec mise en cache mémoire par requête.
    - Couche Services (app/Services/) : 5 services dédiés à la logique métier pure et aux intégrations externes.
    - Contrôleurs (app/Controllers/) : 9 contrôleurs orchestrant la validation des entrées, l'appel aux services/repositories et la transmission des données aux vues.

- Vues (views/) : templates PHP d'affichage découplés de toute requête SQL, avec échappement systématique des sorties (htmlspecialchars).

Bases de données (Architecture polyglotte SQL + NoSQL)

- MySQL 8.0 (Relationnel — PDO) : utilisé pour stocker l'ensemble des données transactionnelles avec contraintes d'intégrité référentielle (FOREIGN KEY, UNIQUE, CHECK) et transactions atomiques (beginTransaction / commit / rollBack).
- MongoDB 7.0 (NoSQL orienté documents) : utilisé via l'extension native ext-mongodb (MongoDB\Driver\Manager et MongoDB\Driver\BulkWrite) pour stocker les documents dénormalisés de statistiques de commandes dans la collection vite_et_gourmand_stats.orders_stats. Ce composant alimente le tableau de bord décisionnel de l'administrateur, avec mécanisme de secours JSON automatique.

Conteneurisation, Versioning et Gestion de projet

- Docker & Docker Compose : conteneurisation complète de l'environnement d'exécution en 3 services isolés (web sous PHP 8.2-Apache, db sous MySQL 8.0, mongo sous MongoDB 7.0).
- Git & GitHub : gestion de versions structurée autour des branches main (production), dev (intégration) et de branches fonctionnelles (feature/*).
- Jira : planification agile, découpage en tickets fonctionnels et suivi de l'avancement du projet.

Déploiement Cloud (Production)

- Heroku : hébergement PaaS de l'application PHP 8.2 / Apache2 (Procfile pointant sur le répertoire sécurisé public/).
- Railway : hébergement Cloud managé de l'instance relationnelle MySQL 8.0 et de l'instance NoSQL MongoDB 7.0 connectées à Heroku par variables d'environnement sécurisées.

2. Configuration de l'environnement de travail et structure du projet

Environnement de développement

- Éditeur de code : Visual Studio Code (extensions PHP Intelephense, Docker, GitLens).
- Environnement local conteneurisé (Docker Desktop) :

    - Conteneur web (Dockerfile basé sur php:8.2-apache) : extensions pdo, pdo_mysql et mongodb compilées et activées, module Apache mod_rewrite activé, DocumentRoot configuré sur /var/www/html/public pour interdire tout accès HTTP direct aux dossiers app/, views/, vendor/ et .env.
    - Conteneur db (mysql:8.0) : initialisé automatiquement au premier démarrage via le script docker/mysql/01-init.sql monté dans /docker-entrypoint-initdb.d/.
    - Conteneur mongo (mongo:7.0) : serveur NoSQL exposé sur le port 27017 avec volume persistant mongo_data.
- Gestionnaire de dépendances : Composer (composer.json) pour l'autoloading PSR-4 (App\ → app/) et la déclaration des extensions requises en production (ext-pdo_mysql, ext-mbstring, ext-mongodb).

Arborescence du dépôt

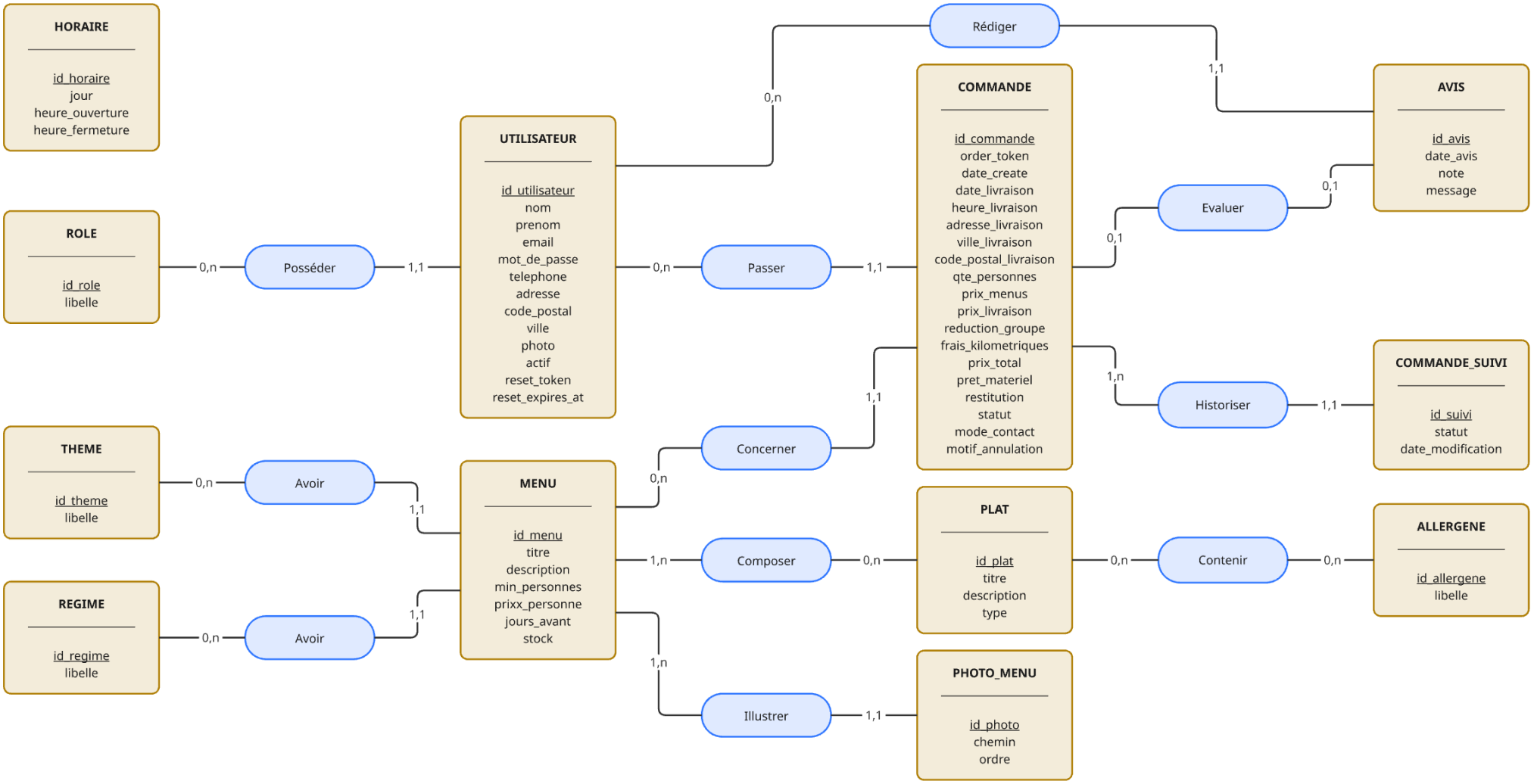
```
vite-et-gourmand/
├── app/
│   ├── Controllers/        # 9 contrôleurs
│   ├── Core/               # 4 composants techniques
│   ├── Models/             # 11 entités métier typées
│   ├── Repositories/       # 11 repositories
│   └── Services/           # 5 services métier
├── docker/
│   └── mysql/
│       ├── 01-init.sql     # Schéma relationnel complet
│       ├── migrations/     # Scripts de migrations SQL
│       └── reset_db.php    # Script CLI de réinitialisation
├── documentation/          # Manuel, Doc technique, Diagrammes
├── public/                 # Seul répertoire exposé par Apache
│   ├── index.php           # Contrôleur frontal unique
│   ├── .htaccess           # Réécriture d'URL vers index.php
│   ├── assets/             # Images, photos de menus et icônes SVG
│   ├── css/                # Feuilles de style et media queries
│   └── js/                 # Scripts JS
├── storage/nosql/          # Persistance documentaire de secours
├── views/                  # Templates de vues PHP
├── .env.example            # Modèle des variables d'environnement
├── compose.yaml            # Orchestration Docker Compose
├── composer.json           # Config Composer, dépendances, autoload
├── Dockerfile              # Image PHP 8.2-Apache personnalisée
└── Procfile                # Config du serveur web sur Heroku
```

Organisation du versioning Git

- main : branche stable de production déployée sur Heroku.
- dev : branche principale d'intégration et de recette.
- Branches feature/* : chaque fonctionnalité majeure a été développée sur une branche dédiée avant d'être fusionnée dans dev puis dans main.

3. Modélisation des données

3.1 Modèle Conceptuel de Données (MCD)

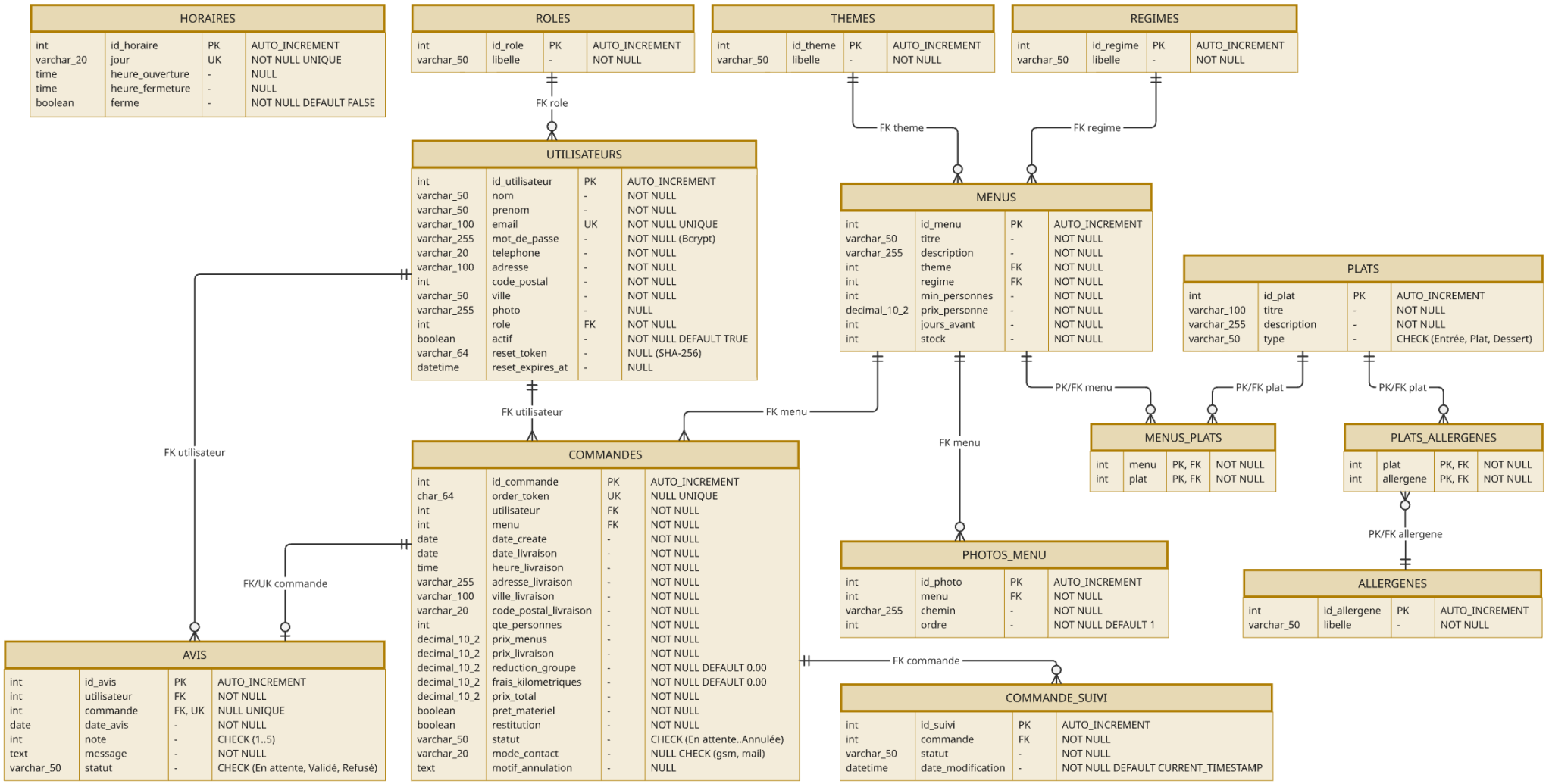


3.2 Modèle Logique de Données (MLD — 3NF)

Le passage du MCD au MLD produit 14 relations (tables) en 3ème forme normale, incluant la migration des clés étrangères (#) pour les relations 1,1 — 0,n et la création des 2 tables associatives MENUS_PLATS et PLATS_ALLERGENES pour les relations n:m :

```
ROLES (id_role, libelle)
UTILISATEURS (id_utilisateur, nom, prenom, mail, password, telephone, adresse_postale, code_postal, ville, actif, reset_token,
reset_expires_at, #role)
THEMES (id_theme, libelle)
REGIMES (id_regime, libelle)
MENUS (id_menu, titre, nb_personne_min, prix_personne, description, conditions, quantite_restante, delai_commande, #theme,
#regime)
PHOTOS_MENU (id_photo, url, legende, #menu)
PLATS (id_plat, titre, description, type)
MENUS_PLATS (#menu, #plat)
ALLERGENES (id_allergene, libelle)
PLATS_ALLERGENES (#plat, #allergene)
COMMANDES (id_commande, order_token, date_create, date_livraison, heure_livraison, adresse_livraison, ville_livraison,
code_postal_livraison, qte_personnes, prix_menus, prix_livraison, reduction_groupe, frais_kilometriques, prix_total,
pret_materiel, restitution, statut, mode_contact, motif_annulation, #utilisateur, #menu)
COMMANDE_SUIVI (id_suivi, statut, date_modification, #commande)
AVIS (id_avis, note, commentaire, date_avis, statut, #utilisateur, #commande)
HORAIRES (id_horaire, jour, ouverture, fermeture, ferme, ordre)
```

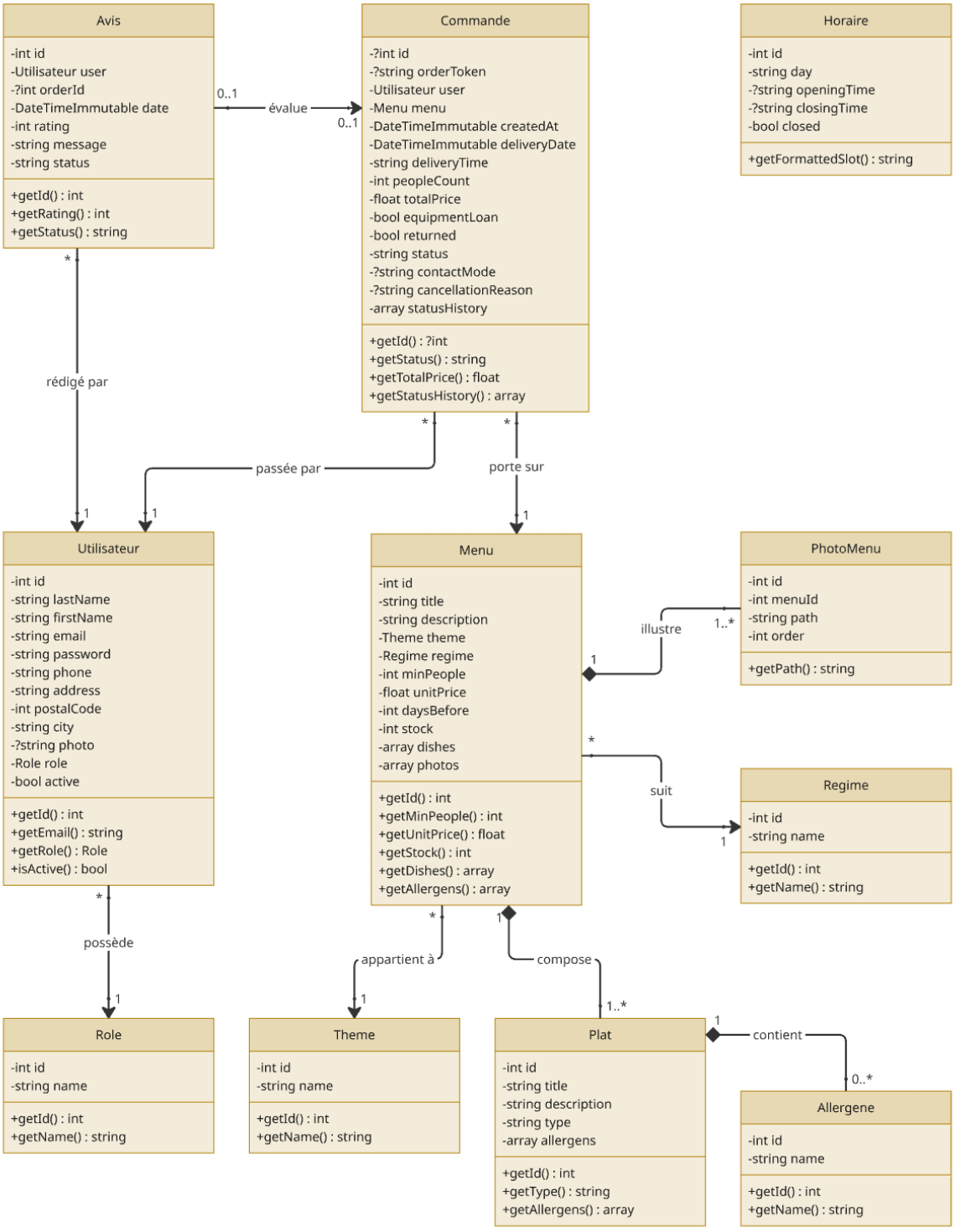
3.3 Modèle Physique de Données



- Sous MySQL (docker/mysql/01-init.sql) : toutes les tables utilisent des contraintes CHECK sur les notes d'avis, les types de plats, les statuts de commande et l'état des comptes.
- Sous MongoDB (StatsNoSqlRepository) : la base vite_et_gourmand_stats stocke dans la collection orders_stats un document JSON par commande contenant _id, order_id, menu_id, menu_titre, prix_personne, date_create, date_livraison, qte_personnes, prix_menus, prix_livraison, reduction_groupe, frais_kilometriques, prix_total, statut et synced_at.

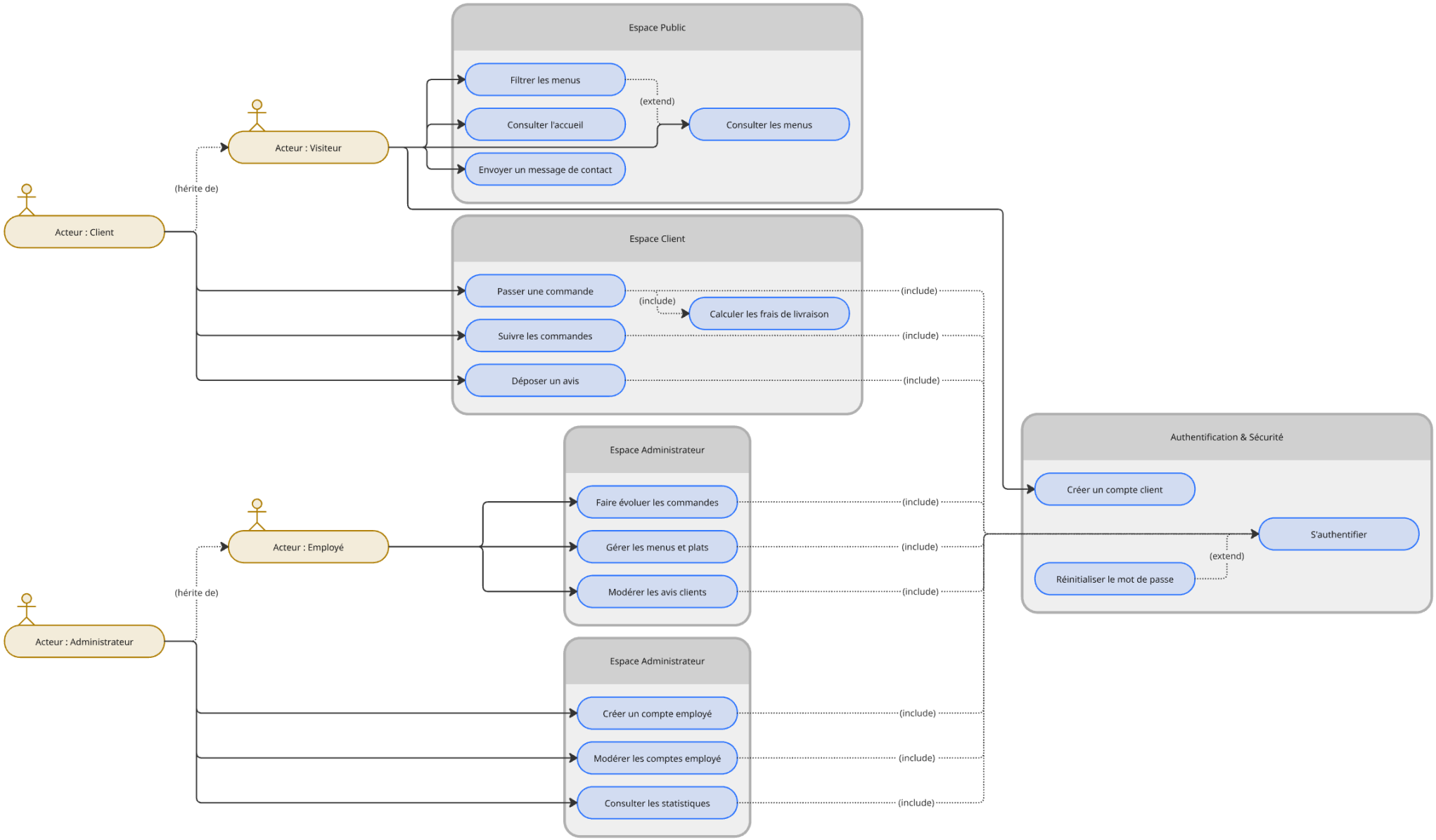
4. Diagramme de classes UML et Architecture MVC / POO

Organisation des couches Orientées Objet



- Couche App\Core :
    - Database : implémente le pattern Singleton par requête HTTP afin de partager une unique instance PDO.
    - Router : enregistre les routes GET et POST et résout l'URI courante vers l'action correspondante.
    - Session : centralise l'authentification en session et le contrôle d'accès par rôle
    - Csrf : génère un jeton et le vérifie en temps constant sur chaque requête POST.
- Couche App\Models : 11 classes métier fortement typées avec relations de composition.
- Couche App\Repositories (Pattern Repository & Optimisation N+1) :
    - Chaque entité dispose de son repository dédié.
    - Optimisation des requêtes N+1 : afin de minimiser la latence réseau entre le serveur applicatif (Heroku) et les bases de données Cloud (Railway), les repositories préchargent en une seule requête SQL les associations, réduisant le nombre de requêtes SQL sur /employe de ~215 à 9 requêtes par page.
- Couche App\Services (Composants métiers) :
    - AuthService : authentification (password_verify), vérification de l'état isActive() du compte et validation de la politique de mot de passe fort (≥10 caractères, 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial).
    - CommandeService : application des règles tarifaires du cahier des charges (remise automatique de −10% dès que convives ≥ min_personnes + 5, livraison gratuite dans Grenoble 38000 ou forfait de 5,00 €+0,59 €/km hors Grenoble) et contrôle du délai minimum de préparation (jours_avant).
    - DistanceService : calcul d'itinéraire routier via l'API Google Routes v2, doublé d'un mécanisme de secours automatique via l'API gouvernementale Base Adresse Nationale (api-adresse.data.gouv.fr) et le moteur routier OSRM pour garantir une disponibilité à 100%.
    - MailService : envoi des notifications par e-mail (bienvenue, confirmation de commande, rappel de restitution de matériel sous 10 jours ouvrés, invitation à déposer un avis, création de compte employé, réinitialisation de mot de passe).

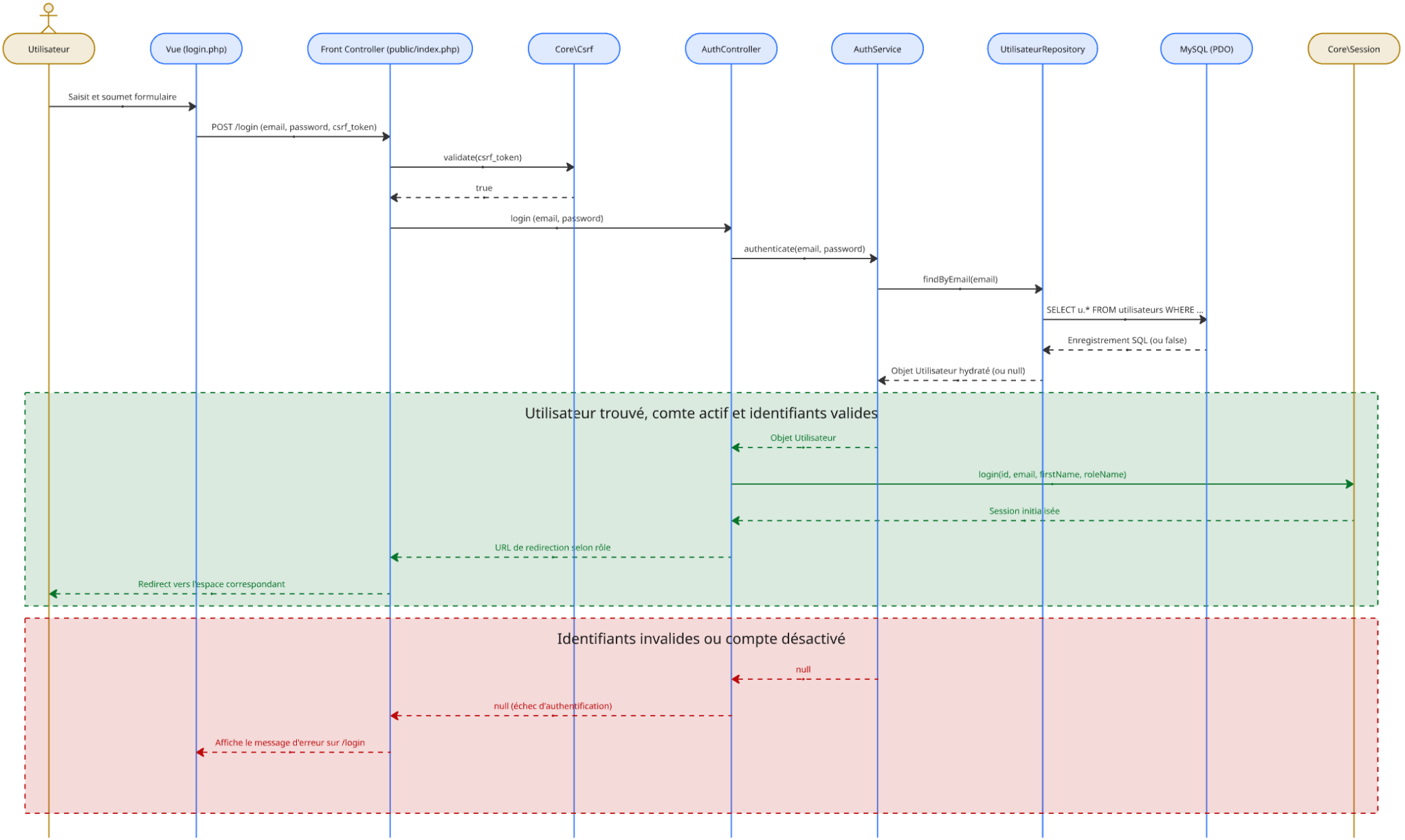
5. Diagramme de cas d'utilisation



Synthèse des droits par acteur

- Visiteur (non connecté) :
    - Consulter l'accueil, les horaires dynamiques et les avis validés.
    - Consulter et filtrer les menus ; consulter la fiche détaillée d'un menu.
    - Envoyer un message via le formulaire de contact.
    - Créer un compte client (/register).
    - Demander la réinitialisation de son mot de passe (/forgot-password).
- Client (Utilisateur connecté) (hérite du Visiteur) :
    - Passer une commande de menu avec calcul automatique de la distance et du tarif (/commande).
    - Modifier ses coordonnées personnelles (/compte).
    - Consulter le suivi horodaté de ses commandes.
    - Modifier (hors menu) ou annuler une commande tant qu'elle est au statut En attente (avec recalcul tarifaire et réajustement automatique du stock).
    - Déposer un unique avis sur une commande au statut Terminée.
- Employé (Employé connecté) :
    - Filtrer et mettre à jour le statut des commandes.
    - Déclencher l'e-mail de rappel de restitution de matériel (pénalité au-delà de 10 jours ouvrés) ou l'invitation à déposer un avis.
    - Annuler une commande en renseignant obligatoirement le mode de contact client et le motif d'annulation.
    - Créer, modifier et supprimer des menus et des plats.
    - Valider ou refuser les avis clients.
    - Modifier les horaires d'ouverture des 7 jours de la semaine.
- Administrateur (Administrateur connecté) (hérite de l'Employé) :
    - Accéder à l'ensemble des fonctionnalités de l'espace Employé.
    - Créer des comptes Employés.
    - Activer ou suspendre un compte Employé.
    - Consulter et filtrer les statistiques de commandes et le chiffre d'affaires par menu issus de la base NoSQL MongoDB.

6. Diagramme de séquence



- Séquence d'authentification MVC (POST /login) : validation du jeton CSRF → AuthController → AuthService → UtilisateurRepository::findByEmail() (requête préparée PDO) → vérification isActive() et password_verify() → initialisation de la session et redirection selon le rôle (/admin, /employe ou /compte).

7. Documentation détaillée du déploiement (Local Docker & Production Cloud Heroku / Railway)

Cette section détaille l'intégralité de la procédure de déploiement de l'application, d'une part en environnement local conteneurisé avec Docker Compose, et d'autre part en environnement de production Cloud (Heroku + Railway MySQL 8.0 + Railway MongoDB 7.0).

7.1 Déploiement en environnement local (Docker & Docker Compose)

A. Prérequis techniques

- Git installé sur le poste hôte.
- Docker Desktop (incluant Docker Engine et Docker Compose v2) actif.
- Ports locaux disponibles : 8080 (serveur web Apache/PHP), 3307 (MySQL 8.0) et 27017 (MongoDB 7.0).

B. Architecture des conteneurs (compose.yaml et Dockerfile)

L'infrastructure locale est orchestrée par le fichier compose.yaml qui déploie trois services interconnectés :

- Service web (construit depuis le Dockerfile) :
    - Image de base : php:8.2-apache.
    - Installation des bibliothèques système (libssl-dev, pkg-config), compilation des extensions pdo, pdo_mysql et installation de l'extension NoSQL mongodb.
    - Activation du module Apache mod_rewrite pour permettre au fichier public/.htaccess de rediriger toutes les routes vers le contrôleur frontal public/index.php.
    - Reconfiguration du DocumentRoot d'Apache vers /var/www/html/public afin d'isoler le code source métier (app/, views/, vendor/, .env) hors de la racine web publique.
- Service db (image officielle mysql:8.0) :
    - Stockage persistant assuré par le volume nommé db_data.

    - Exécution automatique au premier démarrage du script /docker-entrypoint-initdb.d/01-init.sql (monté depuis ./docker/mysql).
- Service mongo (image officielle mongo:7.0) :
    - Stockage persistant assuré par le volume nommé mongo_data.
    - Base documentaire vite_et_gourmand_stats accessible par le conteneur web via l'URI interne mongodb://mongo:27017.

C. Procédure d'installation et de lancement pas à pas

Étape 1 — Cloner le dépôt Git :

```
git clone https://github.com/Mig-Arteaga/vite-et-gourmand.git
cd vite-et-gourmand
```

Étape 2 — Configurer le fichier d'environnement .env : Copier le fichier modèle .env.example vers .env à la racine du projet :

```
cp .env.example .env
```

Variables contenues dans .env :

```
MYSQL_ROOT_PASSWORD=root_secret_password
MYSQL_APP_USER=vg_user
MYSQL_APP_PASSWORD=vg_secret_password
GOOGLE_ROUTES_API_KEY=votre_cle_api_google_optionnelle
```

(Note : si `GOOGLE_ROUTES_API_KEY` est omise, DistanceService bascule automatiquement sur l'API gouvernementale Base Adresse Nationale + OSRM sans aucune interruption de service).

Étape 3 — Construire et démarrer les conteneurs Docker :

```
docker compose up -d --build
```

Étape 4 — Installer les dépendances d'autoloading PSR-4 Composer (si le dossier vendor/ est absent) :

```
docker compose exec web composer install
```

Étape 5 — Vérifier l'état des services et accéder à l'application :

```
docker compose ps
```

L'application est immédiatement accessible à l'adresse http://localhost:8080.

D. Réinitialisation complète de la base de données et synchronisation NoSQL (reset_db.php)

Pour remettre à zéro la base MySQL et resynchroniser immédiatement la collection MongoDB orders_stats à tout moment, un script CLI dédié est fourni :

```
docker compose exec web php docker/mysql/reset_db.php
```

7.2 Déploiement en Production Cloud (Heroku + Railway MySQL & MongoDB)

A. Architecture Cloud de Production

En production, l'application est répartie sur deux plateformes Cloud complémentaires :

- Heroku : exécute le dyno web PHP 8.2 avec le serveur Apache2 et les extensions pdo_mysql, mbstring et mongodb.
- Railway : héberge deux services de base de données distincts :
    - Une instance MySQL 8.0 pour les données relationnelles transactionnelles.
    - Une instance MongoDB 7.0 pour la collection NoSQL vite_et_gourmand_stats.orders_stats.

B. Fichiers de configuration spécifiques au build Heroku

Procfile (à la racine du dépôt) :

```
web: vendor/bin/heroku-php-apache2 public/
```

Cette instruction indique à Heroku de démarrer Apache2 en prenant le sous-dossier public/ comme racine web, protégeant ainsi les répertoires internes du projet.

composer.json :

```
{
    "require": {
        "php": "^8.2",
        "ext-pdo_mysql": "*",
        "ext-mbstring": "*",
        "ext-mongodb": "*"
    },
    "autoload": {
        "psr-4": {
            "App\\": "app/"
        }
    }
}
```

Lors de chaque git push heroku main, le buildpack PHP d'Heroku lit composer.json, installe automatiquement PHP, active les extensions pdo_mysql, mbstring et mongodb, et génère l'autoloader PSR-4 optimisé.

C. Provisionnement des bases de données sur Railway

- Créer un projet sur Railway.app.
- Ajouter le service MySQL 8.0 (+ New → Database → Add MySQL) :
  - Activer l'accès réseau public (TCP Proxy dans l'onglet Settings / Networking) pour obtenir l'hôte et le port publics (MYSQLHOST, MYSQLPORT, MYSQLUSER, MYSQLPASSWORD, MYSQLDATABASE).
- Ajouter le service MongoDB 7.0 (+ New → Database → Add MongoDB) :
  - Activer le TCP Proxy public et récupérer l'URI de connexion publique (MONGO_PUBLIC_URL, de la forme mongodb://mongo:<password>@<host>:<port>).

D. Configuration de l'application et des variables d'environnement sur Heroku

Créer l'application Heroku et lier le dépôt Git :

```
heroku login
heroku create vite-et-gourmand-maao
heroku git:remote -a vite-et-gourmand-maao
```

Configurer les variables d'environnement (Config Vars) sur Heroku : Via le tableau de bord Heroku (Settings → Config Vars) ou en ligne de commande :

```
heroku config:set \
  MYSQLHOST="<hote_public_mysql_railway>" \
  MYSQLPORT="<port_public_mysql_railway>" \
  MYSQLDATABASE="railway" \
  MYSQLUSER="root" \
  MYSQLPASSWORD="<mot_de_passe_mysql_railway>" \

MONGODB_URI="mongodb://mongo:<mot_de_passe_mongo>@<hote_public_mongo
_railway>:<port_mongo>/?authSource=admin" \
  MONGODB_DATABASE="vite_et_gourmand_stats" \
  GOOGLE_ROUTES_API_KEY="<votre_cle_api_google>"
```

La classe App\Core\Database détecte automatiquement ces variables d'environnement (avec compatibilité ascendante entre les conventions Railway MYSQLHOST et Docker DB_HOST), permettant au même code de fonctionner en local et en production sans aucune modification.

E. Déploiement du code et initialisation des bases de production

Pousser la branche main vers Heroku :

```
git push heroku main
```

Initialiser le schéma MySQL et synchroniser MongoDB à distance : Grâce au script CLI docker/mysql/reset_db.php :

```
heroku run php docker/mysql/reset_db.php
```

Sortie attendue :

```
[1/3] Connexion à MySQL réussie.
[2/3] Schéma et données initiales MySQL importés avec succès (01-init.sql).
[3/3] Synchronisation NoSQL terminée : 20 commandes synchronisées dans MongoDB (vite_et_gourmand_stats.orders_stats).
Réinitialisation terminée avec succès !
```

Vérifier l'application en ligne : L'application déployée est accessible en HTTPS à l'adresse :

https://vite-et-gourmand-maao-64b0a66b9b27.herokuapp.com/

8. Sécurité, Optimisation des performances et Tests réalisés

8.1 Mesures de sécurité implémentées

- Protection contre les injections SQL (SQLi) : utilisation exclusive de requêtes préparées PDO (prepare() / execute()) avec désactivation de l'émulation (PDO::ATTR_EMULATE_PREPARES => false) dans les 10 repositories relationnels.
- Protection contre les failles XSS (Cross-Site Scripting) : échappement systématique de toutes les données dynamiques affichées dans les vues via htmlspecialchars($val, ENT_QUOTES, 'UTF-8').
- Protection contre les attaques CSRF (Cross-Site Request Forgery) : génération d'un jeton cryptographique en session (Csrf::generate()) injecté dans tous les formulaires et requêtes fetch POST, vérifié par Csrf::validate() via hash_equals().
- Hachage et politique des mots de passe : hachage systématique via password_hash(..., PASSWORD_DEFAULT) (Bcrypt), vérification via password_verify(), obligation d'un mot de passe fort conforme aux recommandations CNIL et jetons de réinitialisation à usage unique hachés en SHA-256 avec expiration à 1 heure.
- Contrôle d'accès basé sur les rôles (RBAC) : verrouillage serveur de chaque route sensible (Session::requireRoles(['Employé', 'Administrateur']) sur /employe/*,

Session::requireRoles(['Administrateur']) sur /admin/*, vérification de propriété utilisateur = :user_id sur toutes les actions client /compte/*, et blocage immédiat de la connexion si un compte employé est désactivé actif = 0).

- Protection contre la double soumission de commande : génération d'un jeton d'idempotence order_token doté d'une contrainte UNIQUE en base de données.

8.2 Tests réalisés avant et après déploiement

- Validation syntaxique automatisée (php -l) : vérification de l'intégralité des 65 fichiers PHP du projet (app/, public/, views/, docker/mysql/reset_db.php) — 0 erreur de syntaxe détectée.
- Tests fonctionnels par rôle (Visiteur, Client, Employé, Administrateur) :
    - Parcours Visiteur : affichage des horaires dynamiques et des avis validés, filtrage combiné des menus sans rechargement, envoi du formulaire de contact.
    - Parcours Authentification : création de compte client avec test de rejet des mots de passe faibles, demande de réinitialisation et changement de mot de passe via jeton temporaire, connexion/déconnexion et blocage d'un compte employé suspendu.
    - Parcours Commande & Espace Client : pré-remplissage des coordonnées, calcul de distance routière et application de la remise de -10% dès min_personnes + 5, réservation transactionnelle du stock, modification et annulation d'une commande En attente, dépôt d'un avis unique sur une commande Terminée.
    - Parcours Employé : filtrage des commandes, progression des statuts et alimentation de commande_suivi, annulation avec motif et mode de contact obligatoires, CRUD complet des menus et des plats, modération des avis clients, mise à jour des horaires d'ouverture.
    - Parcours Administrateur : création d'un compte employé, activation/suspension en un clic, filtrage des statistiques et vérification de la cohérence du chiffre d'affaires agrégé depuis MongoDB.
- Tests d'accessibilité RGAA et Responsive Design :
    - Vérification du zoom texte jusqu'à 200% (absence de user-scalable=no), navigation au clavier (Tab, Escape sur les modales), présence des attributs alt, aria-label, title sur l'<iframe> de carte et association <label for="..."> sur tous les champs de formulaire.